

Day 4 PostgreSQL 18 Backup and Disaster Recovery Lab Manual

Fully corrected dry run edition | Zalando Postgres Operator and Patroni | Namespace **postgres-ha** | Cluster **acid-postgres-ha**

Follow the sections in order. Commands are for a Bash terminal with kubectl access. Copy the code only, without a prompt. A line ending in a backslash continues on the next line. Replace only values explicitly called placeholders. Every gate states the expected result before you proceed.

Scope: logical and physical backup, Docker Hub image, GCS repository, mounted pgBackRest configuration, WAL archiving, full, incremental and differential backups, and an isolated restore. Restore never targets the active Patroni PVCs. The separate Ubuntu Kubernetes cluster is not GKE; a GCS bucket alone does not grant its pods access.

0 Start here and check the running cluster

Run once per new terminal. The shell variables keep the following commands short. **echo** only prints its arguments: echo "\$NS" displays the value; it does not change a file or a Kubernetes object. Re-run this block after opening a fresh terminal.

```
NS=postgres-ha
CLUSTER=acid-postgres-ha
IMAGE=docker.io/eswaribala/ericpostgrerepo2026:pg18
echo "$NS $CLUSTER $IMAGE"
kubectl -n "$NS" get postgresql "$CLUSTER"
kubectl -n "$NS" get pods
kubectl -n "$NS" get sts "$CLUSTER" \
-o jsonpath='{.spec.updateStrategy.type}'{"\n"}
```

Expected: three database pods **acid-postgres-ha-0, -1, -2** Ready; update strategy may say **OnDelete**. Do not include PgBouncer pooler pods in database checks. Never run kubectl rollout status on this OnDelete StatefulSet; it reports an unsupported strategy even when all three pods are Ready.

Identify the current primary through the Patroni REST API. Re-run after failover; pod 0 is not guaranteed to be the primary.

```
PRIMARY=
for p in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do
  role=$(kubectl -n "$NS" exec "$p" -c postgres -- \
    curl -fsS http://127.0.0.1:8008/ | jq -r .role)
  echo "$p $role"
  case "$role" in primary|leader|master) PRIMARY="$p"; esac
done
echo "Primary: $PRIMARY"
test -n "$PRIMARY"
```

Stop if no primary was found. The banking database must exist for the example database dump below; otherwise replace banking with a database that exists in your cluster.

1 Lab 1 Logical backup

pg_dump creates one database dump; pg_dumpall includes roles and all databases. Output redirection with > writes a file on **your Ubuntu computer**. It is not stored in the pod or GCS bucket.

```
mkdir -p backups/logical
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
```

```
pg_dump -U postgres -Fc banking > backups/logical/banking.dump
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
pg_dumpall -U postgres > backups/logical/all-databases.sql
ls -lh backups/logical
pg_restore -l backups/logical/banking.dump | head -25
```

Expected: both files are nonempty and `pg_restore` lists objects. Do not restore over the live banking database.
Optional restore check: create a different disposable database and restore through standard input.

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
psql -U postgres -c 'CREATE DATABASE banking_restore_test;'
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
pg_restore -U postgres -d banking_restore_test --no-owner \
< backups/logical/banking.dump
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
psql -U postgres -d banking_restore_test -c '\dt'
```

If `banking_restore_test` already exists, choose another new database name consistently in all commands. A custom dump can require extensions or existing roles.

2 Lab 2 Physical base backup

This independent exercise uses the standby replication secret. Confirm the secret name from `kubectl` output; it can vary by operator configuration. The primary Service `acid-postgres-ha` routes to the current leader.

```
kubectl -n "$NS" get secret | grep standby
kubectl -n "$NS" get svc acid-postgres-ha
REPL_SECRET=standby.acid-postgres-ha.credentials.postgresql.acid.zalan.do
REPL_USER=$(kubectl -n "$NS" get secret "$REPL_SECRET" \
-o jsonpath='{.data.username}' | base64 -d)
REPL_PASS=$(kubectl -n "$NS" get secret "$REPL_SECRET" \
-o jsonpath='{.data.password}' | base64 -d)
REPLICA=acid-postgres-ha-1
test "$REPLICA" != "$PRIMARY" || REPLICA=acid-postgres-ha-2
kubectl -n "$NS" exec "$REPLICA" -c postgres -- df -h /tmp
```

Only continue if `/tmp` has sufficient free space. Some clusters require `sslmode=require` for replication; the example sets it explicitly. Do not paste `REPL_PASS` into screenshots.

```
kubectl -n "$NS" exec "$REPLICA" -c postgres -- \
env PGPASSWORD="$REPL_PASS" PGSSLMODE=require \
pg_basebackup -h acid-postgres-ha -p 5432 \
-U "$REPL_USER" -D /tmp/day4-basebackup -Ft -z -Xs -P
mkdir -p backups/physical
kubectl -n "$NS" cp -c postgres \
"$REPLICA:/tmp/day4-basebackup" backups/physical/day4-basebackup
unset REPL_PASS
find backups/physical/day4-basebackup -maxdepth 2 -type f
```

If authentication fails, inspect `pg_hba` and the replication role. Do not change `pg_hba` to `trust`. `pg_basebackup` files in `/tmp` are temporary and must be copied out.

3 Lab 3 Check existing WAL archive before any change

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
psql -U postgres -c 'SHOW archive_mode;'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
psql -U postgres -c 'SHOW archive_command;'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
psql -U postgres -x -c 'SELECT * FROM pg_stat_archiver;'
```

Record these results. Spilo may already archive with WAL-G or WAL-E. Do not disable a working archive until the new `pgBackRest` repository is mounted, reachable and tested; keep the old repository while validating the new

backup chain.

4 Lab 4 Verify the custom Docker image

The actual image name contains **ericpostgrerepo2026**, with the letter **s** after ericpo. The misspelling ericpotgrerepo2026 caused ImagePullBackOff. The image must contain the pgbackrest executable; image name alone does not prove this.

```
kubectl -n "$NS" get postgresql "$CLUSTER" \
-o jsonpath='{.spec.dockerImage}{"\n"}'
kubectl -n "$NS" get sts "$CLUSTER" \
-o jsonpath='{.spec.template.spec.containers[?(@.name=="postgres")].image}{"\n"}'
for p in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do
  kubectl -n "$NS" get pod "$p" \
  -o custom-columns='NAME:.metadata.name,IMAGE:.spec.containers[0].image,READY:.status.containerStatuses[0].ready'
  kubectl -n "$NS" exec "$p" -c postgres -- pgbackrest version
done
```

Expected: the CR, StatefulSet template and each pod show the exact custom image; three pgbackrest version commands work and readiness is true. If all pass, **do not edit or delete pods again**. If the image is wrong, change **spec.dockerImage** in kubectl edit postgresql acid-postgres-ha. The metadata.name and PostgreSQL stanza name remain unchanged. A message "Edit cancelled, no changes made" means that particular edit saved nothing. Verify the CR and StatefulSet again before recreating one outdated replica at a time. With OnDelete, recreation happens only after a pod is deleted; never delete the primary or several members to force an update.

5 Lab 5 Prepare the Google Cloud Storage bucket

Run gcloud commands from the Ubuntu machine where you are logged in. gcloud auth login selects an account; gcloud config set project selects a **PROJECT_ID**. The project must have active billing before Cloud Storage can create a bucket.

```
gcloud auth list
gcloud projects list
gcloud config get-value project
gcloud billing projects describe "$(gcloud config get-value project)"
```

If billing is disabled, enable it for the intended project in the Google Cloud Billing console before continuing. Do not substitute a project name for PROJECT_ID. Assign the actual project and a globally unique bucket name below; the example name is usable only if creation succeeds.

```
PROJECT_ID=$(gcloud config get-value project)
GCS_BUCKET=postgres-ha-backups-parameswari-2026
gcloud storage buckets create "gs://$GCS_BUCKET" \
--project="$PROJECT_ID" --location=asia-south1 \
--default-storage-class=STANDARD \
--uniform-bucket-level-access --public-access-prevention
gcloud storage buckets describe "gs://$GCS_BUCKET"
```

If it already exists under your project, the describe command is the verification. If create returns HTTP 403 and billing account disabled, stop and fix billing. **GCS_BUCKET is a shell variable**: GCS_BUCKET=... assigns it; echo "\$GCS_BUCKET" prints it; unset GCS_BUCKET removes it from this terminal only.

6 Lab 6 Give the Ubuntu Kubernetes pods access to GCS

The database pods are running in your Ubuntu Kubernetes cluster, not necessarily on GKE. This manual uses a Google service account JSON key for this training cluster. It must be kept private. If your organization prohibits key creation, use a configured workload identity mechanism instead and set repo1-gcs-key-type=auto; do not use auto without verified pod identity. Bucket access and Docker Hub pull credentials are different.

Run on Ubuntu, after the bucket exists. The first command creates a service account in the selected project. If the account already exists, skip only that command. Restrict the Object Admin role to the backup bucket, not the whole project.

```
GSA=pgbackrest-day4
GSA_EMAIL="$GSA@$PROJECT_ID.iam.gserviceaccount.com"
gcloud iam service-accounts create "$GSA" \
  --project="$PROJECT_ID" --display-name='Day 4 pgBackRest'
gcloud storage buckets add-iam-policy-binding "gs://$GCS_BUCKET" \
  --member="serviceAccount:$GSA_EMAIL" \
  --role=roles/storage.objectAdmin
umask 077
gcloud iam service-accounts keys create ./pgbackrest-gcs-key.json \
  --iam-account="$GSA_EMAIL" --project="$PROJECT_ID"
kubectl -n "$NS" create secret generic pgbackrest-gcs-key \
  --from-file=key.json=./pgbackrest-gcs-key.json
kubectl -n "$NS" get secret pgbackrest-gcs-key
```

Keep the JSON key in a secure store for disaster recovery and remove the unprotected local working copy when that is complete. A Kubernetes Secret is not automatically mounted: section 8 explicitly mounts it. Avoid displaying the key contents in output.

7 Lab 7 Create the pgBackRest encryption Secret and ConfigMap

The Secret name is **pgbackrest-repo-secret**. Its key is **repo-cipher-pass**; the path in the container will be **/etc/pgbackrest-secret/repo-cipher-pass**. Do not change the file path to **/etc/pgbackrest-repo-secret** merely because the Kubernetes Secret has that name.

```
PGDATA=$(kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -Atqc 'SHOW data_directory;')
echo "$PGDATA"
read -rsp 'Choose and save an encryption passphrase: ' CIPHER_PASS
echo
test -n "$CIPHER_PASS"
kubectl -n "$NS" create secret generic pgbackrest-repo-secret \
  --from-literal=repo-cipher-pass="$CIPHER_PASS"
unset CIPHER_PASS
kubectl -n "$NS" get secret pgbackrest-repo-secret
```

Backups encrypted with this passphrase cannot be restored without it. Preserve a secure recovery copy outside this Kubernetes cluster. A second invocation of `kubectl create secret` can return `AlreadyExists`; inspect the existing Secret instead of generating a different passphrase.

Create `pgbackrest.conf` in your Ubuntu working directory using the actual PGDATA and bucket. The service key setting matches the non-GKE authentication method in section 6.

```
cat > pgbackrest.conf <<EOF
[postgres-ha]
pg1-path=$PGDATA
pg1-port=5432

[global]
repol-type=gcs
repol-gcs-bucket=$GCS_BUCKET
repol-gcs-key-type=service
repol-gcs-key=/etc/pgbackrest-gcs/key.json
repol-path=/pgbackrest/postgres-ha
repol-retention-full=2
repol-cipher-type=aes-256-cbc
compress-type=zst
process-max=2
log-level-console=info
EOF
kubectl -n "$NS" create configmap pgbackrest-config \
```

```
--from-file=pgbackrest.conf=./pgbackrest.conf
kubectl -n "$NS" get configmap pgbackrest-config
```

If the ConfigMap already exists, update its content with `kubect create configmap ... --dry-run=client -o yaml | kubectl apply -f -` rather than leaving stale settings. Do not put the encryption passphrase in this ConfigMap.

8 Lab 8 Mount all three resources through the operator

Creating the ConfigMap and Secrets does **not** make files appear in pods. The operator supports `spec.additionalVolumes` on recent releases. First check your installed CRD, then edit the PostgreSQL custom resource and add these three entries under the existing **spec:**. Keep all other spec entries. If `additionalVolumes` already exists, append the entries to that list. The block below is YAML to place in the editor, **not a terminal command**.

```
kubectl explain postgresql.spec.additionalVolumes
kubectl -n "$NS" edit postgresql "$CLUSTER"
```

```
additionalVolumes:
- name: pgbackrest-config
  mountPath: /etc/pgbackrest
  volumeSource:
    configMap:
      name: pgbackrest-config
- name: pgbackrest-repo-secret
  mountPath: /etc/pgbackrest-secret
  volumeSource:
    secret:
      secretName: pgbackrest-repo-secret
- name: pgbackrest-gcs-key
  mountPath: /etc/pgbackrest-gcs
  volumeSource:
    secret:
      secretName: pgbackrest-gcs-key
```

In the editor, put these lines indented beneath `spec:`. Save in nano with Ctrl+O, Enter, Ctrl+X; in vim, press Esc, then type `:wq` and Enter. If `kubectl edit` says Edit cancelled, the new configuration was **not** saved. Inspect the custom resource and generated StatefulSet; the operator can take a short time to reconcile.

```
kubectl -n "$NS" get postgresql "$CLUSTER" \
-o jsonpath='{.spec.additionalVolumes}{"\n"}'
kubectl -n "$NS" get sts "$CLUSTER" \
-o jsonpath='{.spec.template.spec.containers[?(@.name=="postgres")].volumeMounts}{"\n"}'
kubectl -n "$NS" get sts "$CLUSTER" \
-o jsonpath='{.spec.template.spec.volumes}{"\n"}'
```

Expected in the StatefulSet template: three paths `/etc/pgbackrest`, `/etc/pgbackrest-secret` and `/etc/pgbackrest-gcs`, and three matching ConfigMap or Secret volumes. If the template lacks them, do not restart pods; inspect `kubectl describe postgresql` and operator logs first.

OnDelete update: Existing pods keep their prior mounts until they are recreated. First identify the primary again with the section 0 role commands. Recreate **one Ready replica at a time** after validating the template and pull credentials. Watch the pod return to Ready before touching the next member. Never delete the primary as a routine mount change; arrange a controlled switchover or operator maintenance procedure after both replicas are healthy.

```
kubectl -n "$NS" get sts "$CLUSTER" \
-o jsonpath='{.spec.updateStrategy.type}{"\n"}'
kubectl -n "$NS" get pods \
acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2
```

For each selected **replica only**, replace `POD_NAME` with its exact current name, then run the command once. If you are unsure which is primary, return to section 0.

```

POD_NAME=acid-postgres-ha-1
if test "$POD_NAME" = "$PRIMARY"; then
    echo 'STOP: selected pod is the primary'
else
    kubectl -n "$NS" delete pod "$POD_NAME"
    kubectl -n "$NS" wait --for=condition=Ready \
        "pod/$POD_NAME" --timeout=10m
fi

```

The three-file check must run only after the three pods have received the new template. It produces a separate OK or MISSING result for each file; unlike the old loop, it identifies the failing file. Never print the secret or key value.

```

for p in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do
    echo "=== $p ==="
    kubectl -n "$NS" exec "$p" -c postgres -- \
        test -r /etc/pgbackrest/pgbackrest.conf \
        && echo 'config OK' || echo 'config MISSING'
    kubectl -n "$NS" exec "$p" -c postgres -- \
        test -r /etc/pgbackrest-secret/repo-cipher-pass \
        && echo 'cipher OK' || echo 'cipher MISSING'
    kubectl -n "$NS" exec "$p" -c postgres -- \
        test -r /etc/pgbackrest-gcs/key.json \
        && echo 'GCS key OK' || echo 'GCS key MISSING'
done

```

Stop until all nine checks print OK. A Secret listed by kubectl get secret can still be absent in a pod that lacks its mount.

9 Lab 9 Create stanza and configure WAL archiving

The primary must be current. Test repository access before modifying archive_command. The pgBackRest info command can report missing stanza initially; it should not report missing config, missing key, 403, or billing disabled.

```

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc \
    'export PGBACKREST_REP01_CIPHER_PASS=$(cat /etc/pgbackrest-secret/repo-cipher-pass);
    pgbackrest --stanza=postgres-ha stanza-create'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc \
    'export PGBACKREST_REP01_CIPHER_PASS=$(cat /etc/pgbackrest-secret/repo-cipher-pass);
    pgbackrest --stanza=postgres-ha info'

```

Configure archiving only after a successful stanza-create. Zalando stores PostgreSQL parameter overrides under **spec.postgresql.parameters** in its custom resource. Edit the same PostgreSQL resource, adding archive_mode and archive_command alongside any existing parameters. archive_mode=on needs a PostgreSQL restart; schedule a controlled restart and verify each member. Existing WAL-G or WAL-E configuration needs deliberate migration.

```

kubectl -n "$NS" edit postgresql "$CLUSTER"

```

This is a fragment within the existing spec.postgresql object, not a complete manifest. Preserve version and all existing parameters:

```

postgresql:
  version: '18'
  parameters:
    archive_mode: 'on'
    archive_command: >-
      PGBACKREST_REP01_CIPHER_PASS=$(cat /etc/pgbackrest-secret/repo-cipher-pass)
      pgbackrest --stanza=postgres-ha archive-push %p

```

If an operator overwrites this archive_command, pause and inspect its existing WAL archive mechanism; do not repeatedly change a live cluster by hand. After the controlled PostgreSQL restart and checking the actual primary,

verify the effective settings, switch WAL and run check.

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -c 'SHOW archive_mode;\'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -c 'SHOW archive_command;\'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -c 'SELECT pg_switch_wal();\'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc \
  'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)";
  pgbackrest --stanza=postgres-ha check'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -x -c 'SELECT * FROM pg_stat_archiver;\'
```

Expected: archive_mode on, archive_command shows pgbackrest, check succeeds, and archived_count advances without repeated failures. A newly created stanza is not a completed backup.

10 Lab 10 Full, incremental and differential backups

Run from the current primary. A full backup is the base. After a test change, incremental records changes since the latest backup. Differential records changes since the latest full. Do not assume a fixed pod ordinal remains primary after a failover.

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc \
  'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)";
  pgbackrest --stanza=postgres-ha --type=full backup'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -d banking -c "CREATE TABLE IF NOT EXISTS dr_day4_demo
  (id int primary key, note text);"
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -d banking -c "INSERT INTO dr_day4_demo VALUES
  (1, 'after full') ON CONFLICT (id) DO UPDATE SET note=excluded.note;"
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc \
  'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)";
  pgbackrest --stanza=postgres-ha --type=incr backup'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -d banking -c "INSERT INTO dr_day4_demo VALUES
  (2, 'after incremental') ON CONFLICT (id) DO UPDATE SET note=excluded.note;"
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc \
  'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)";
  pgbackrest --stanza=postgres-ha --type=diff backup;
  pgbackrest --stanza=postgres-ha info'
```

Expected in pgbackrest info: full, incremental and differential backup labels. Check the bucket objects with gcloud storage ls "gs://\$GCS_BUCKET/pgbackrest/postgres-ha/". Keep the encryption passphrase and credentials required for recovery.

11 Lab 11 Restore safely on a separate PVC

Create a **new** PVC used only by a recovery toolbox; do not mount a PVC belonging to acid-postgres-ha-0, -1, or -2. This section verifies a physical restore into an offline directory. It does not claim that the restored PostgreSQL has started; a complete recovery drill must additionally launch PostgreSQL 18 from that separate PVC, inspect recovery logs and query the data.

```
kubectl -n "$NS" get pvc
cat > day4-recovery-pvc.yaml <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: day4-recovery-data
  namespace: $NS
spec:
  accessModes: [ReadWriteOnce]
```

```

resources:
  requests:
    storage: 30Gi
EOF
kubectl apply -f day4-recovery-pvc.yaml
kubectl -n "$NS" get pvc day4-recovery-data

```

If the PVC is Pending, choose the storage class available in your cluster; stop here until Bound. Use a size appropriate for the real database. The following toolbox inherits the service account of a PostgreSQL pod; it also mounts the private Docker Hub pull Secret and all pgBackRest resources.

```

POD_SA=$(kubectl -n "$NS" get pod "$PRIMARY" \
-o jsonpath='{.spec.serviceAccountName}')
PG_UID=$(kubectl -n "$NS" exec "$PRIMARY" -c postgres -- id -u)
PG_GID=$(kubectl -n "$NS" exec "$PRIMARY" -c postgres -- id -g)
cat > day4-recovery-toolbox.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: day4-recovery-toolbox
  namespace: $NS
spec:
  restartPolicy: Never
  serviceAccountName: $POD_SA
  imagePullSecrets:
    - name: dockerhub-secret
  securityContext:
    runAsUser: $PG_UID
    runAsGroup: $PG_GID
    fsGroup: $PG_GID
  containers:
    - name: toolbox
      image: $IMAGE
      command: ["sh", "-c", "sleep 36000"]
      volumeMounts:
        - {name: data, mountPath: /recovery}
        - {name: config, mountPath: /etc/pgbackrest}
        - {name: cipher, mountPath: /etc/pgbackrest-secret}
        - {name: gcs, mountPath: /etc/pgbackrest-gcs}
  volumes:
    - name: data
      persistentVolumeClaim: {claimName: day4-recovery-data}
    - name: config
      configMap: {name: pgbackrest-config}
    - name: cipher
      secret: {secretName: pgbackrest-repo-secret}
    - name: gcs
      secret: {secretName: pgbackrest-gcs-key}
EOF
kubectl apply -f day4-recovery-toolbox.yaml
kubectl -n "$NS" wait --for=condition=Ready \
pod/day4-recovery-toolbox --timeout=10m

```

Before restoring, confirm the toolbox is using day4-recovery-data and that the data path is empty. The pg1-path override changes the target from the live PGDATA in the ConfigMap to a dedicated recovery location. If the image stores PostgreSQL configuration elsewhere, startup needs a separate recovery procedure.

```

kubectl -n "$NS" get pod day4-recovery-toolbox \
-o jsonpath='{.spec.volumes[?(@.name=="data")].persistentVolumeClaim.claimName}'
kubectl -n "$NS" exec day4-recovery-toolbox -c toolbox -- \
sh -c 'test ! -e /recovery/data/PG_VERSION'
kubectl -n "$NS" exec day4-recovery-toolbox -c toolbox -- sh -lc \
'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)";
pgbackrest --stanza=postgres-ha --pg1-path=/recovery/data restore'
kubectl -n "$NS" exec day4-recovery-toolbox -c toolbox -- \
cat /recovery/data/PG_VERSION

```

Expected: PG_VERSION displays 18. This proves files were restored. To prove application recovery, start a separate isolated PostgreSQL 18 instance using this data directory and the correct WAL recovery settings. Do not connect the original operator StatefulSet or its active Services to this PVC.

12 Lab 12 Point in time recovery practice

PITR needs a completed full backup and archived WAL spanning the target timestamp. Make a disposable test change, record an exact UTC target after the good transaction and before the bad one, and switch WAL. Use **a second new empty recovery PVC** and change the toolbox claimName to that PVC before attempting another restore. Never rerun restore into a running database or over the previous restore directory.

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -d banking -c "INSERT INTO dr_day4_demo VALUES
  (3, 'good') ON CONFLICT (id) DO UPDATE SET note=excluded.note;"
TARGET_TIME=$(kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -Atqc "SELECT to_char(clock_timestamp()
  AT TIME ZONE 'UTC', 'YYYY-MM-DD HH24:MI:SS.US') || '+00';")
echo "$TARGET_TIME"
sleep 3
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -d banking -c "DELETE FROM dr_day4_demo WHERE id=3;"
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -c 'SELECT pg_switch_wal();'
```

Make a second empty recovery PVC and point a new instance of the recovery toolbox at it. These commands delete only the disposable toolbox Pod, **not** the first recovery PVC or any Patroni pod. Confirm that the new claim is Bound and the toolbox shows day4-pitr-data before running PITR.

```
sed 's/day4-recovery-data/day4-pitr-data/g' \
  day4-recovery-pvc.yaml > day4-pitr-pvc.yaml
kubectl apply -f day4-pitr-pvc.yaml
kubectl -n "$NS" get pvc day4-pitr-data
sed 's/day4-recovery-data/day4-pitr-data/g' \
  day4-recovery-toolbox.yaml > day4-pitr-toolbox.yaml
kubectl -n "$NS" delete pod day4-recovery-toolbox
kubectl apply -f day4-pitr-toolbox.yaml
kubectl -n "$NS" wait --for=condition=Ready \
  pod/day4-recovery-toolbox --timeout=10m
kubectl -n "$NS" get pod day4-recovery-toolbox \
  -o jsonpath='{.spec.volumes[?(@.name=="data")].persistentVolumeClaim.claimName}{"\n"}'
```

The final output must say day4-pitr-data. TARGET_TIME is the value recorded above; do not leave it blank. Verify WAL archiving completed before proceeding. After restore, start an isolated PostgreSQL server to verify row 3 exists and the later delete has not been applied.

```
test -n "$TARGET_TIME"
kubectl -n "$NS" exec day4-recovery-toolbox -c toolbox -- \
  env TARGET_TIME="$TARGET_TIME" sh -lc \
  'test ! -e /recovery/data/PG_VERSION;
  export PGBACKREST_REP01_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)";
  pgbackrest --stanza=postgres-ha --pg1-path=/recovery/data \
  --type=time --target="$TARGET_TIME" --target-action=promote restore'
```

This PITR command is for a fresh toolbox attached to a new empty PVC. If the earlier full restore is still present, test -e fails and prevents overwriting it. An offline restore is not a completed PITR drill until database startup and SQL verification pass.

13 Error guide and final checklist

Edit cancelled: kubectl edit saved nothing; check the custom resource and edit again. **Old image name:** correct spec.dockerImage, wait for the generated StatefulSet template, then replace only one replica at a time under OnDelete. **rollout status error:** OnDelete does not support that command; use kubectl get pods and readiness. **Exit code 1 on test -r:** check the exact missing path and pod volumeMounts. **Secret exists but file missing:** add spec.additionalVolumes and recreate pods in a controlled order. **403 billing disabled:** fix billing on the project owning the bucket. **GCS 403 access denied:** inspect the service account identity and bucket IAM. **Backup succeeds but restore fails:** check encryption passphrase, retained backup chain and archived WAL.

Completion criteria: three healthy PostgreSQL pods have the same pgBackRest image and three readable resources; the actual GCS bucket exists and its key can access it; pgBackRest stanza-create and check pass; WAL archiving progresses; full, incremental and differential sets appear in info; a separate recovery PVC receives PostgreSQL 18 files; a standalone isolated database startup and SQL check complete the recovery drill.

Authoritative references: Zalando Postgres Operator cluster manifest: https://postgres-operator.readthedocs.io/en/latest/reference/cluster_manifest/ ; pgBackRest configuration: <https://pgbackrest.org/configuration.html> ; Google Cloud bucket creation: <https://docs.cloud.google.com/storage/docs/creating-buckets> . Review commands against the installed operator and pgBackRest versions before using them outside this training cluster.